# Microservices with Java and Spring Boot

Jan Ypma

November 1, 2026

# Outline

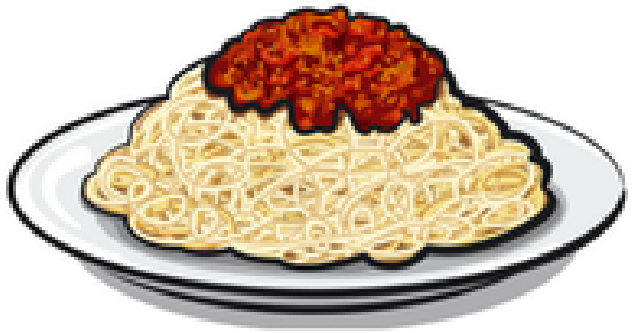

1. Introduction
   - Getting started
   - What's a microservice
   - The 8 fallacies of distributed computing
2. Design for resilience
   - Service failure
   - Creating services
   - Consuming services
   - Guidelines
3. Break(out) 1
   - Service discovery
4. Infrastructure architecture
   - It's a linux world
   - Partitioned data stores
   - Single-server data stores
   - Monitoring and alerting
   - Request tracing
   - Deployment
   - Configuration
   - Load balancer
5. Break(out) 2
   - Demo project
   - Alternative: Infrastructure discovery
6. Data architecture
   - Domain-driven design
   - Data design patterns
   - Data formats
7. Break

THE EVOLUTION OF

# SOFTWARE ARCHITECTURE

## 1990's

SPAGHETTI-ORIENTED
ARCHITECTURE
(aka Copy & Paste)



## 2000's

LASAGNA-ORIENTED
ARCHITECTURE
(aka Layered Monolith)



Lund&Bendsen

*developing developers*

Slides available here: https://github.com/jypma/LB3210

- What you're seeing here i literally in the file `LB3210.org` (rendered OK by Github)
- Have you already cloned that repository? Great! Please, `git pull`
- PDF of slides will be available at completion

# About your instructor

- Jan Ypma
  - Almost 24 years of Java experience (yay, generics!)
  - Java, Scala, C++, Rust, Lisp
  - Contributor to `akka` (Apache Pekko), `akka-http` and others
  - Functional programming and distributed systems
  - Agile coach and fan of Domain-Driven Design
  - `jan@ypmania.net`, `https://linkedin.com/in/jypma`
- Warning: Subjectivity ahead!

# Schedule, day 1

| Time | Duration | Activity | Weight |
|---|---|---|---|
| 09:00 | 00:10 | Welcome, Outline/Agenda | 2 |
| 09:10 | 00:20 | Round table introductions | |
| 09:30 | 00:10 | What's a microservice | 4 |
| 09:40 | 00:20 | Design for resilience | 12 |
| 10:00 | 00:30 | Break(out) 1: Service discovery | |
| 10:30 | 00:20 | Discussion of breakout results | |
| 10:50 | 00:45 | Infrastructure architecture | 27 |
| 11:35 | 00:20 | Break(out) 2: Project setup | |
| 11:55 | 00:20 | Data architecture (DDD, Patterns) | 13 |
| 12:15 | 00:30 | Lunch | |
| 12:45 | 00:20 | Data architecture (Formats) | |
| 13:05 | 00:45 | Software architecture | 28 |
| 13:50 | 00:10 | Demo project API demo and code overview | 5 |
| 14:00 | 00:55 | Break(out) 3: Extend project | |
| 14:55 | 00:30 | Presentation(s) of extended demo project | |
| 15:25 | 00:35 | Wrap-up, reserved time for extra subjects | |

# Talk about yourself

- My Background
- What do I hope to get out of the course
- What's the smallest service I've ever written

# Definition

- *Service*
  - One operating system process (often on its own server)
  - Exposes an API (sometimes also a UI)
- *Micro*
  - Theory: It's small
  - Practice: There are many
  - Independently deployable

# Philosophy

- Business needs evolve
- Team composition changes
- Services should be disposable (design to be replaceable)
  - Rebuilt in 1-3 months
- Per service, use best technology matching experience and requirements

# Service scope

- Service belongs to one team
  - Team is responsible for entire service software life cycle
- Data store belongs to one service
- Independently deployable

# Use cases

- Embrace Conway's law: One system belongs to at most one team
- Monoliths are fine to start with
  - Time to market and technical debt vs. holistic design
- Strangler pattern

# Which of these is true?

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

source

# How well does the following abstraction help?

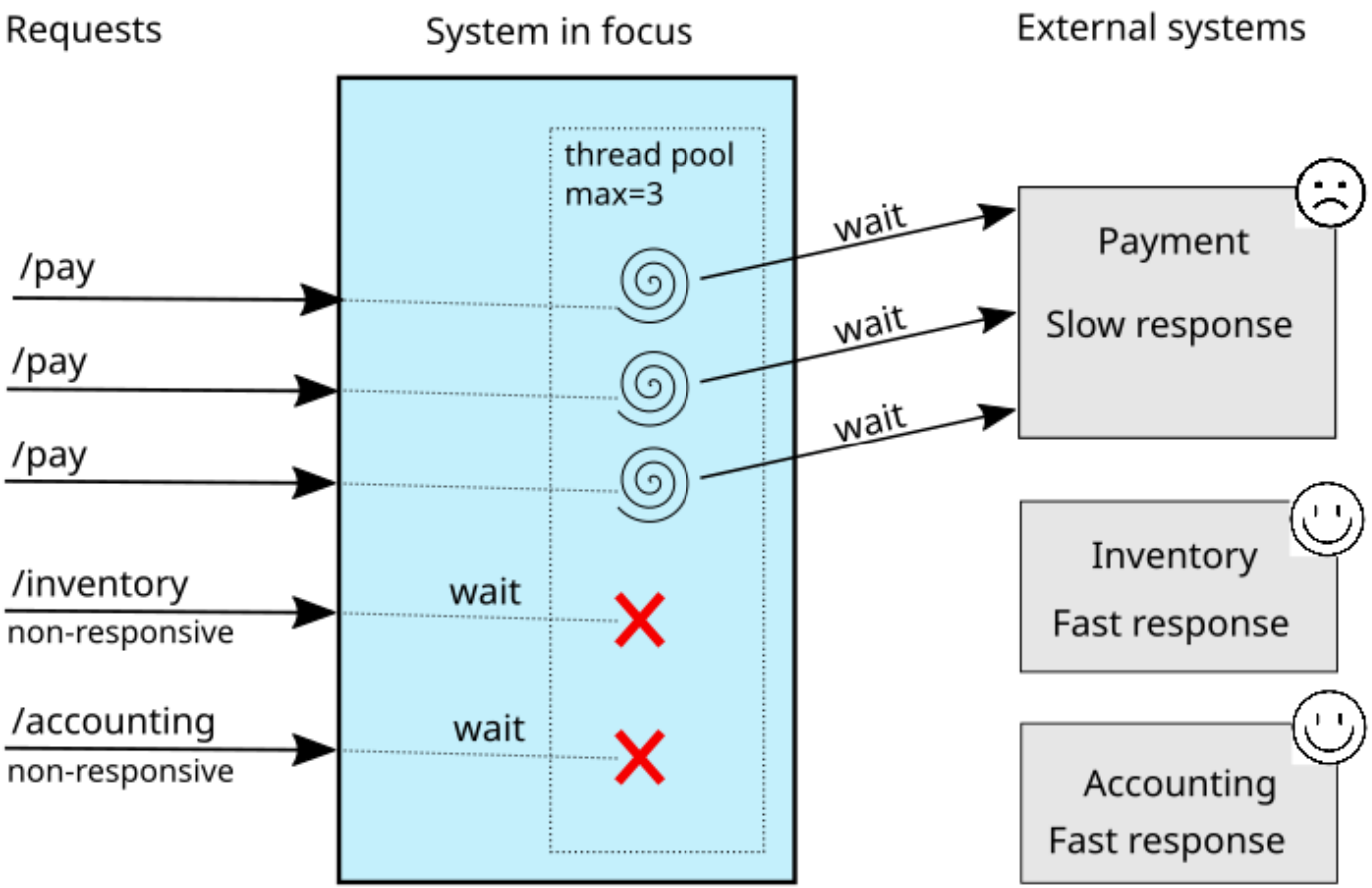
```
MyResponseType callMyRemoteService(String command, byte[] data)
```

# Your (or your colleague's) service will be down

P(everything working) = P(one service is working) ^ $n_{services}$
*Our service is up 99% of the time!*
Well, we have about 30 microservices, each with 3 copies. That means that 63% of the time, at least one service is down somewhere.

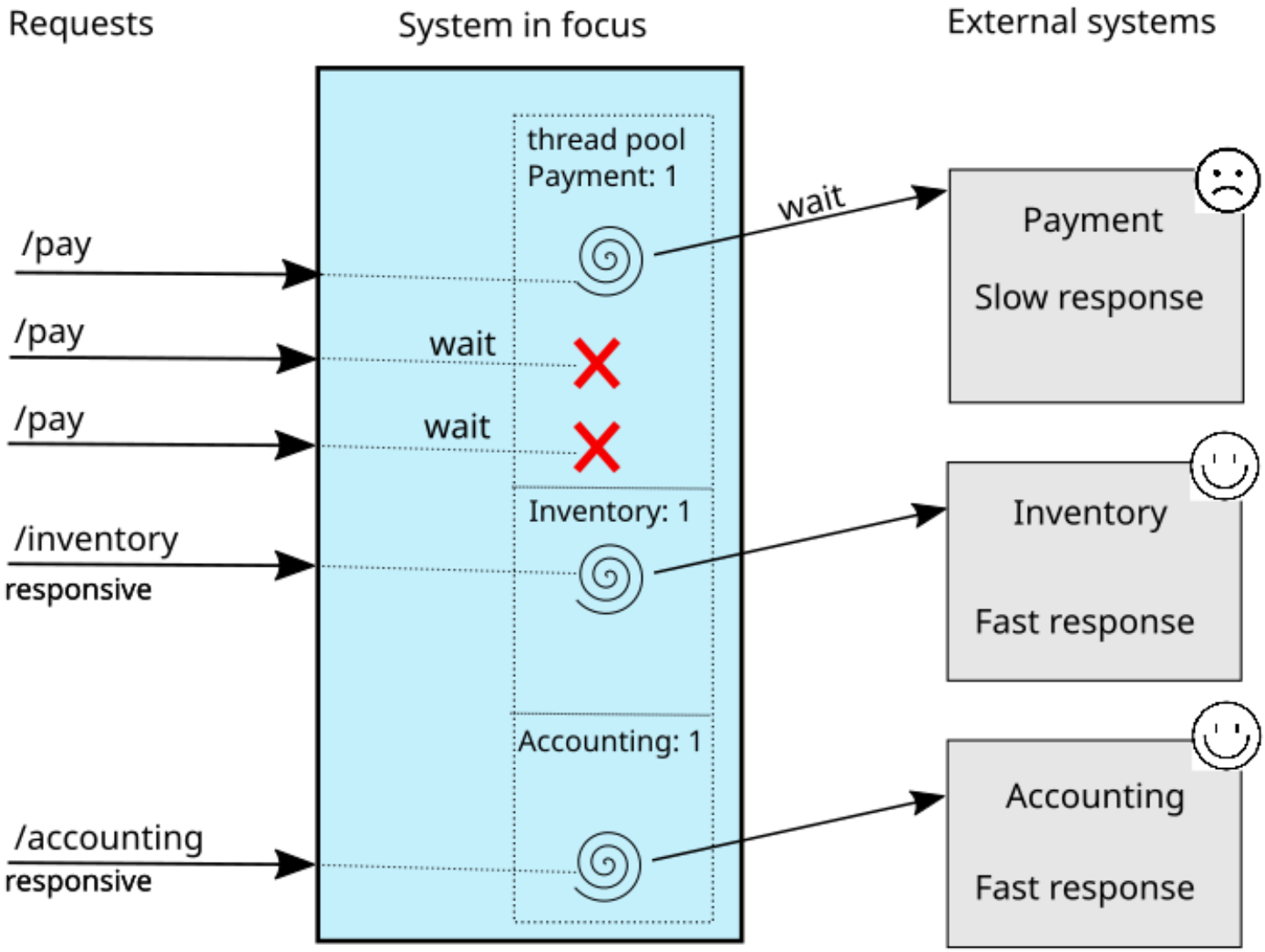
# Bulkhead

A single resource pool is covering multiple types of application requests



# Bulkhead in place

- Semaphore with an optional timed queue in front
- Other parts of the resource pool are still accessible



# Measure service quality

- Service Level Indicator (SLI)
  - An actual number that indicates the current value of an *SLO*, e.g.
    - 99th percentile response time
    - 24-hour window success rate of HTTP requests
- Service Level Objective (SLO)
  - Metric that indicates a healthy service to you, e.g.
    - *"The 99th percentile of HTTP response times is at most 300ms"*
    - *"At least 99.9% of HTTP requests result in a successful response"*
  - Typically only internally measured and/or agreed between teams
- Service Level Agreement (SLA)
  - Part of a contractual obligation (sometimes legally binding) between parties
    - *"The 95th percentile of HTTP response times is at most 1000ms"*
    - *"At least 99% of HTTP requests result in a successful response"*
  - Typically results in a stricter SLO being applied internally

# Recommendations

When creating microservices:

- Prefer sharded (partitioned) data stores over single points of failure
- Embrace eventual consistency
  - Idempotency for all incoming data
- Always deploy more than 1 copy
  - Investigate the need for a cluster-aware distributed framework
- Have a *Service dashboard* with metrics
- Use `Bulkhead` to protect finite resources

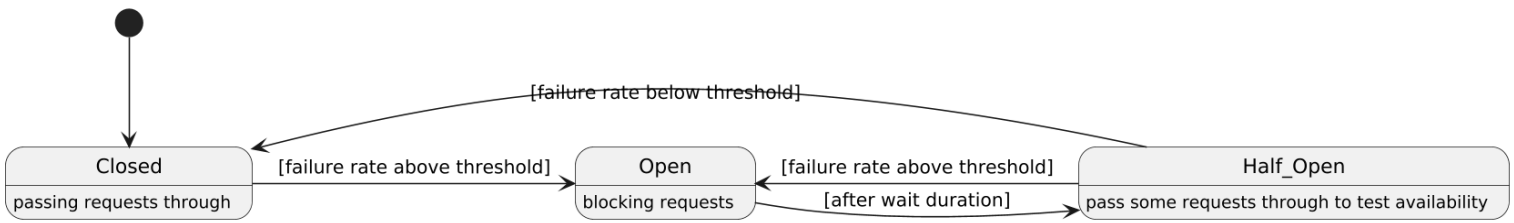
More on all of these later.

# Guidelines

- Design for failure
  - Have methods/functions reflect doing I/O
  - Make time (and timeouts) explicit
  - Use `Circuit Breaker` where applicable
- Fail fast
  - `System.exit(1)` is a viable error handler

# Circuit breaker

- Smart state machine towards 1 backend
  - *Closed*: Everything is working normally
  - *Open*: We've determined that the backend is down, and block requests
  - *Half-open*: We're allowing a few requests through, to test the waters



# Microservice pitfalls

- Service co-dependencies
  - Keep HTTP calls one way only
  - Plugin pattern
- Nested synchronous service calls
  - API gateway calls backend, which calls calculation service, which calls user service, which calls authorization service
  - Added latency and failure possibility
  - Avoid these with event sourcing
  - Alternatives:
    - Change service boundaries
    - Replicate data instead
    - Call asynchronously

# Need more inspiration?

- The twelve-factor app, https://12factor.net/
  - Provides sensible suggestions on a lot of topics
    - Port binding, dev/prod differences, admin processes
  - Not the only way (geared towards ruby/python), but worth a thorough read
- Self-contained systems: `https://scs-architecture.org/`

# Introduction

- Break into teams of 2-4 people (20 min)
- Discuss the services and projects you've been a part of, and identify:
  - Examples of a microservice
  - Examples of *definitely NOT* a microservice
- For each service found, describe how *resilient* the given service was
  - Usage or absence of *bulkhead* and/or *circuit breaker*
  - Usage or absence of clustering / replication
- Oh, and check your laptop, make sure you have the following:
  - `git`
  - `docker-compose`
  - `maven`
  - `java` (21+)

# Discussion

- Describe the services you have found


EXCUSE ME!


Do you have a moment to talk about Linux?

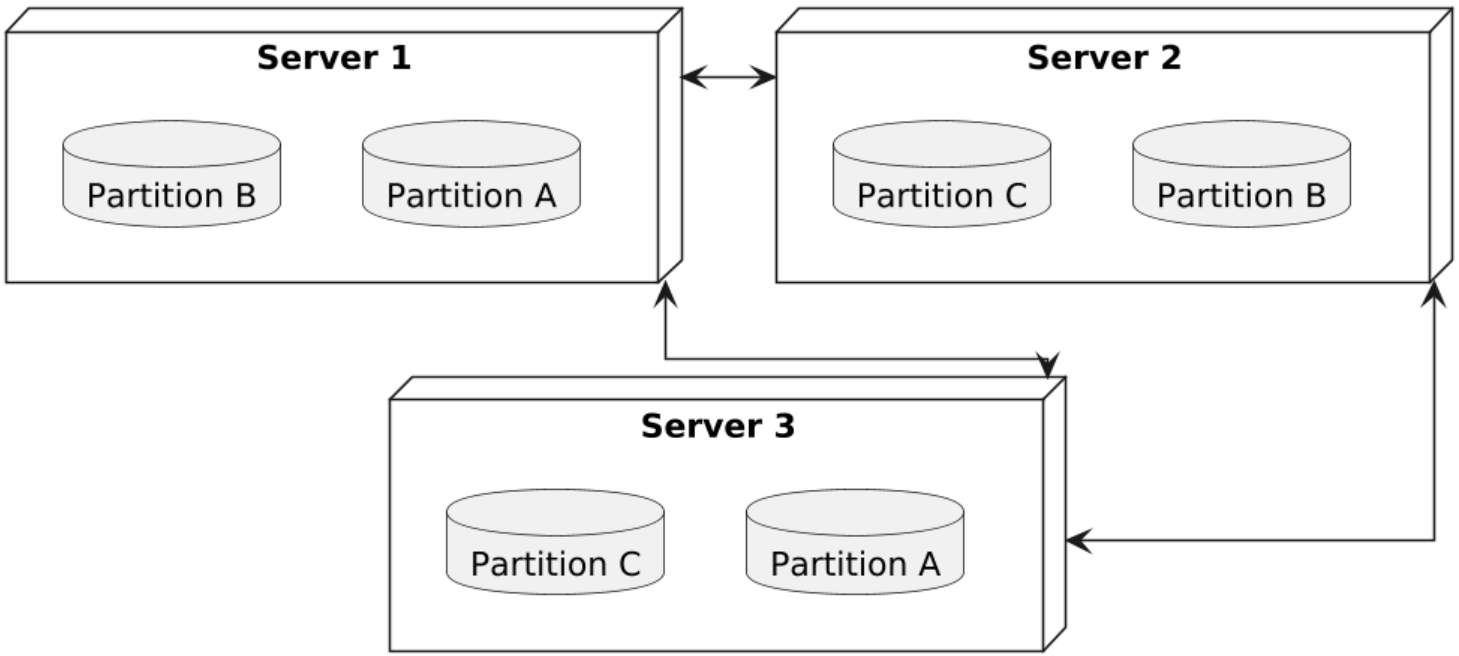
# Get familiar with linux

- Micro services are a linux world
- It's easier than ever to get started
  - WSL 2 (some integration, traditionally has had issues, probably better today)
  - VirtualBox with e.g. Ubuntu (real linux)
  - Dual boot e.g. Ubuntu
  - Just get a Raspberry Pi



# Partitioned data stores: introduction

- All data is split into partitions (also called *shards*), which are copied onto servers
- For each data element, a *key* determines which partition it's stored on



# Partitioned row stores

Each *row* has a *key* that specifies which partition(s) store data for that row. Data is typically stored in columns, following a schema.

- Open source: Cassandra
- Amazon: DynamoDB, Keyspaces
- Google: BigTable
- Azure: Cosmos DB (with Cassandra API)

# Example cassandra queries

- Creating a table

```
CREATE TABLE chat_messages (
  roomId int,
  seqNr int,
  edited timestamp,
  userId int,
  message text,

  PRIMARY KEY (roomId, seqNr)
);
```

- Table must have a primary key
- Part of the primary key is the *partition* key, which dictates how the data is partitioned (sharded)
- Inserting (or updating) rows

```
INSERT INTO chat_messages (roomId, seqNr, edited, userId, message)
  VALUES (1, 1, NOW(), 42, 'This is my message');
```

- This will insert (or overwrite) the row for the data's primary key values
- `UPDATE` also exists, and has the same semantics
- Did somebody say this is NoSQL?

# Partitioned queues

Messages sent to a queue (sometimes called topic) are distributed to partitions, based on a *key*. Messages typically small (some services have upper limit of 64kB).

- Open source: Kafka
- Amazon: SQS
- Google: Cloud Pub/Sub
- Azure: Storage Queue ( * ) , Service Bus ( * )

( * ) *not partitioned, size-limited*

# Partitioned search

Full-text search is often important when dealing with data.

- Open source: Elasticsearch, SoLR
- Amazon: Hosted elasticsearch
- Google: Hosted elasticsearch
- Azure: Hosted elasticsearch

# Single-server data stores: introduction

- Many moving parts needed to make primary/replica failover work
  - PostgreSQL: Multiple servers possible, but failures leak to the client. `pgBouncer` as alternative.
  - MariaDB: Multiple servers possible with failover, fail-back is a manual process
  - RabbitMQ: Multiple servers possible with failover
- If you choose these, make failover and failback testing part of your CI

# RabbitMQ

- Message queue with focus on performance
- Original architecture single-server
  - Later extended with *Mirror Queues* (primary/replica)
  - Extended with *Quorum Queues* in 2019 (based on raft), regularly expanded since:
    - Limited message TTL
    - All cluster members have all data
    - Complete message index in memory

# RabbitMQ Data consistency

- AMQP "transaction"
  - Covers only a single queue
  - "Slow" (fsync for every transaction)
- *Publisher confirms*
  - Asynchronous message from RabbitMQ to client (after fsync): `basic.ack` or `basic.nack`
  - Lost broker connection: risk duplicate, or risk lost messages
- Manual *Consumer acknowledgement*
  - Consumer sends message to RabbitMQ to confirm handling of message is complete
  - `basic.ack`, `basic.nack(requeue)`, `basic.nack(no requeue)`
  - This is async, so no guarantee that the server receives it
    - Two generals agree

- Logging need not be a cross-cutting concern
  - Create monitored metrics instead
- Your service dashboard is as important as your public API
  - Have metrics on *everything*
  - Dashboard should be visible to and understandable by non-team members
- Be aware of your resource usage, check all environments at least daily

# Protocol variations

- Push-based (statsd)
  - Application periodically (10 seconds) sends UDP packet(s) with metrics
  - Simple text-based wire format
  - Composes well if running with multiple metrics backends
  - Advantages: composability, easy to route, less moving parts
- Pull-based (prometheus)
  - Database calls into microservice periodically (10 seconds) over HTTP
  - Service needs to run extra HTTP server
  - Does not compose (multiple metrics backends need to be known on the prometheus side)
  - Advantages: less timing-sensitive
- Most popular solution today: Prometheus
  - Natively pull-based
  - Has many connectors to allow push-based systems to contribute metrics

# Metrics terminology

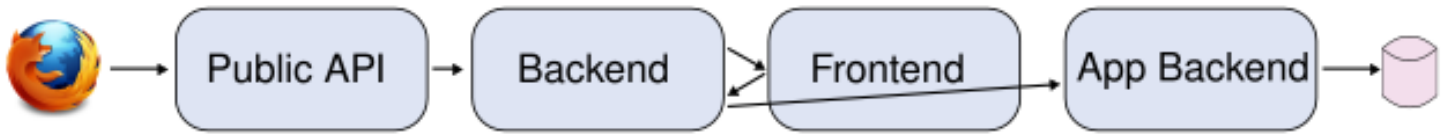
- Different frameworks use different terms
- Micrometer uses the following:
  - *Counter* (sometimes called *event*): An occurrence of a discrete event
    - e.g. a request coming in
  - *Gauge*: The size of a single measurable quantity (and its unit)
    - e.g. the number of active TCP connections
  - *Timer*: The duration of an activity
    - e.g. the response time to a request
  - *Distribution summary* (sometimes called *histogram* or even *gauge*): Recorded values (and units) that go with events
    - e.g. the size of incoming requests in bytes

# Example dashboard

- https://play.grafana.org/

# Complex service dependencies

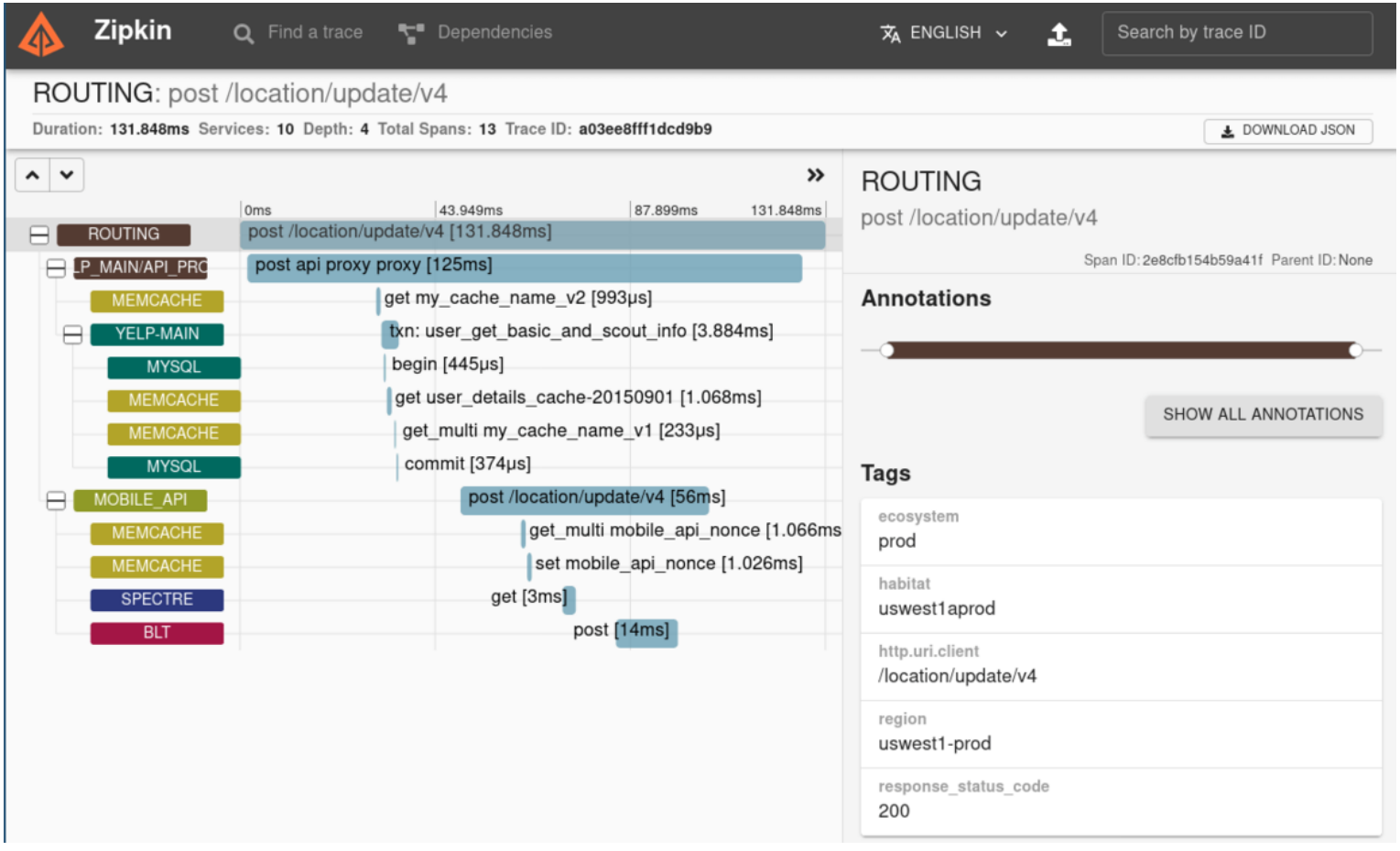
- Services can have complex calling stacks




- When something breaks, it's good to have a trace
- Other reasons
  - Identify performance problems
  - Find bottlenecks
  - Track resource usage

# Two mature solutions

- Jaeger and Zipkin
  - Both have vast library and framework support
  - Many metrics framework support both backends



# Hosted, semi-hosted or self-hosted?

- Learning a new data store technology
  - Reliability guarantees
  - Scalability and performance characteristics
  - API
  - Life cycle management and backup of data
  - Installation and operation (for developers)
  - Installation and operation (in production)
- You can save on the last bullet, but not on the others
- Self-hosted
  - You install and run everything yourself
    - Kafka, Cassandra, Elasticsearch
    - Typically on Docker & Kubernetes
  - Can re-use knowledge and code between development and production
- Semi-hosted
  - Cloud provider installs and operates existing (typically open source) software for you
  - But you still have to pick server size and count
  - You're billed per server
- Hosted
  - Cloud provider installs and operates everything for you
  - You're billed per logical storage unit (e.g. database row or queue message)

# Pets vs. cattle

- *Pets*: Traditional server management
  - Servers have cute names
  - Some server names I've seen: `pinkie`, `oink`, `tardis`, `deepthought`, `zeus`
  - Everyone know the peculiarities of each server
- *Cattle*: Cloud server management
  - Servers have only a logical ID or number
  - Hardware setup, rack and/or location
  - Find an available server to put your service on

# Virtualization and containerization

- First, there was plain hardware
- VM abstraction
  - Decoupling of multiple roles of one server
  - Memory and disk overhead
  - Linux optimizations (kernel shared memory)
- Linux can do many of this natively
  - *Namespaces*: Hide processes from each other
  - *Cgroups*: Limit resource usage
- Containers to make it fast and efficient
  - VM: GBs
  - Docker (ubuntu): 100's of MB
  - Docker (alpine): MBs
  - Instant startup

- Limited to linux in this course
- Lightweight layer over native cgroups isolation
- Dockerfile

```
FROM node:12-alpine
RUN apk add --no-cache python g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
```

- Layers
- Volumes
  - Handling of persistent data
- Port mapping
- User mapping
- Don't run as root

# Docker-compose

```yaml
version: '3.1'

services:

  db:
    image: postgres:13.2-alpine
    # Uncomment this to have the DB come up when you start docker / your
    ↪  laptop:
    #restart: always
    environment:
      POSTGRES_USER: demo
      POSTGRES_DB: demo
      POSTGRES_PASSWORD: example
    ports:
      - 5432:5432

  rabbitmq:
    image: rabbitmq:3.8.16-alpine
    # Uncomment this to have the DB come up when you start docker / your
    ↪  laptop:
    #restart: always
    ports:
      - 5672:5672    # AMQP
      - 15672:15672  # Web UI
```

- Groups several docker containers and storage
- Ideal for local testing

# Kubernetes

- Manages a cluster of distributed docker containers with copies
  - *Pod*: Combination of one or more docker containers and their configuration
  - *Configmap*: Extra settings for pods, typically becoming a volume in the pod
  - *Deployment*: Automatic replicas and distributed upgrades for pods (and other resources)
- Ideal for production
- Configure Memory requests and limits (compatible with your heap size)
- Configure CPU requests
- Get comfortable getting thread and heap dumps
- Heap dump on out of memory (this *will* happen)
  - `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps` to an `emptyDir` volume

# Handling of externalized values

- Externalize "magic numbers" and strings into configuration files
- Use your framework to differentiate between *internal* and *external* configuration
  - *Internal* (inside docker container) has defaults and values that don't really change
  - *External* (mounted as a volume) has settings specific for that environment and/or server
- Changes to configuration files
  - Kubernetes: Configmap change does *not* restart the pod
  - Hot reloading? Not in spring boot (watch file and shutdown instead)
- Environment variables for secrets: don't do it (leaking to docker, monitoring tools)
  - use files instead
- Environment variables for service injection: don't do it (ordering issues)
  - use DNS instead (e.g. `dns-java`, `pekko discovery`, [...])

# Allowing the world to call your service

- Deployed kubernetes services only reachable within the cluster
- Need to define an `ingress`
  - HTTP-level (NGinx or Kong, "API gateway") or TCP-level (HAProxy)
  - Provided by your native cloud provider or kubernetes
  - Hybrid setups
- Additional, external, load balancer in front of ingress controller
- Pick the simplest setup that satisfies your needs

# Get the demo project

This course, and its accompanying Spring Boot demo project, are on github.

- Make sure you have the prerequisites installed:
  - `git`
  - `docker-compose`
  - `maven`
  - `java` (21+)
- Get the course and code

```
git clone git@github.com:jypma/LB3210.git
```

*OR*

```
git clone https://github.com/jypma/LB3210.git
```

- Try to run the demo project

```
cd LB3210/demo-project
docker-compose up -d
./mvnw spring-boot:run
```

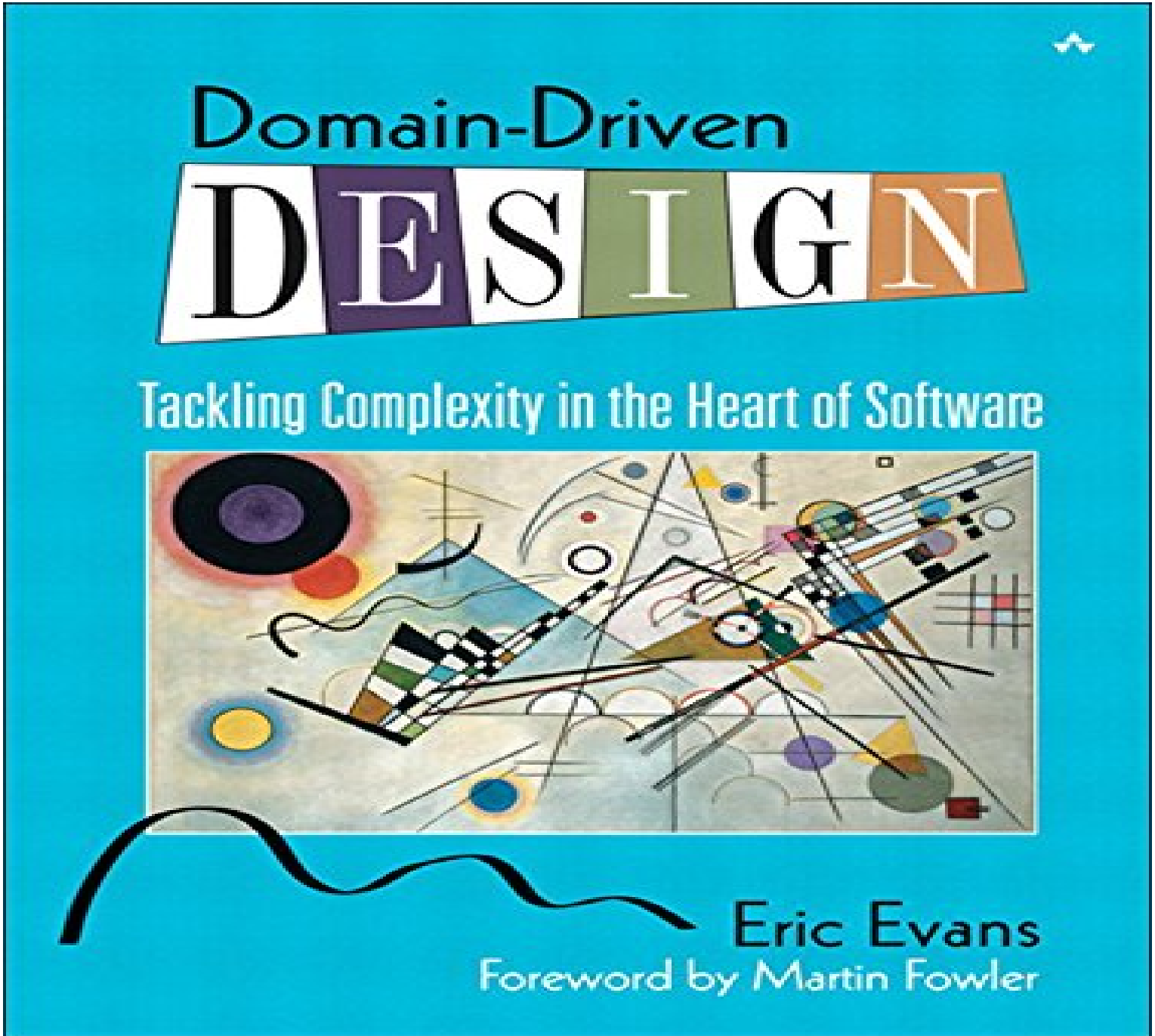
- If you have time, get the code into your favorite IDE and browse around it

- Resume in your teams
- Which pieces of infrastructure exist around the services you discovered?
- Who "owns" or maintains them?
- How can you set up new infrastructure?
- Look at all categories of infrastructure:
  - Servers
  - Data stores
  - Load balancers and gateways
  - Monitoring and dashboards
  - Others

# Discussion

- Describe the infrastructure you have found

# The book


Domain-Driven
DESIGN
Tackling Complexity in the Heart of Software
Eric Evans
Foreword by Martin Fowler

# Introduction

- Software methodology
  - *Names in code must names used by the business*
- Popularized in 2003 by Eric Evans in his book
- Simple guideline lead to extremely useful patterns
- Useful example here

# Bounded context

- Reasoning about complex business processes requires abstractions
  - A *domain model* implements these abstractions as code
- Abstractions, and hence models, have a limited applicability
- *Bounded context* makes this explicit
  - When creating a domain model, evaluate the scope of your design
  - Create sub-domains when you encounter them
  - Describe the bounds for your domain
- Bounded context is often a good candidate for Microservice boundaries

# Ubiquitous language

- We have a domain model, great!
- Added value comes from day-to-day conversations
  - Among developers
  - Between developers and the customer
  - Between developers and the user
- Is everyone speaking the same language?
- *Ubiqutous language*: All team members use important terms in the same way
  - Within a bounded context

# Implementation patterns

DDD defines several patterns that are relevant as software architecture during implementation.

- *Entity*: A mutable representation of one business concept
- *Value Object*: An immutable set of data going to or from an *Entity*
- *Aggregate*: A group of *Entities* that should always have a consistent view

Have you implemented similar patterns? What did you call them?

# Event storming workshop

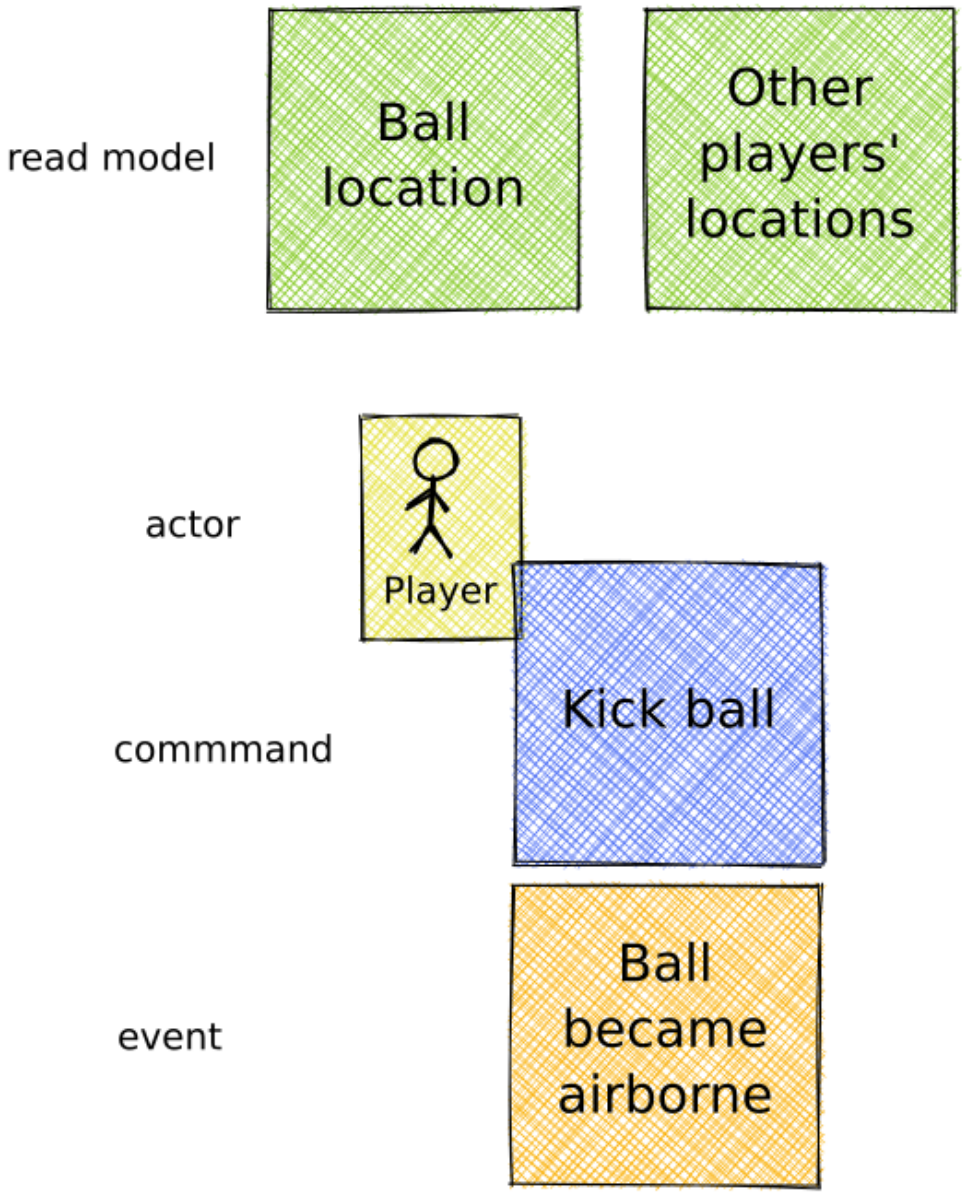
- We need to quickly learn a new domain
  - *Business process modeling* and *requirements gathering*
  - Bring together *domain experts* and *developers*: Invite the right people!
  - Primary goal is a mutual understanding of the domain
- Alberto Brandolini (2012): Event Storming

# Event storming elements

- Discover events that occur in the business, and what triggers them
  - Business Event, e.g. *a customer has applied for a loan*
    - A change has occurred (in your business or in the real world)
  - Command, e.g. *create a new loan request*
    - A request or interaction to be made with a system (ours or external)
    - Decided upon and initiated by a user, or by an external system
  - Read model, e.g. *customer account balance*
    - Information that a user or external system needs to base commands on
  - Actor, e.g. *loan requester*
    - Role of a person submitting a command
  - Aggregate, e.g. *Loan Application*
    - Entity(ies) of a business subdomain that should be viewed atomically

# Event storming workshop (example)

- An small example of all concepts is here



# Idempotency

- Allow any failed or repeated action to be applied again
  - With the same result (if previously successful)
  - Without additional side effects that have business impact
- Example:
  - New user is stored in our database, but afterwards we failed sending their welcome mail (SMTP server was down).
    - Retry the database operation: User is already found, so instead we verify that the data matches
    - Retry sending the mail: We know that we didn't send the mail yet, so we send it once more
  - New user is stored, welcome mail is sent, but we failed updating our CRM system
    - Retry the database operation: User is already found, so instead we verify that the data matches
    - Retry sending the mail: We know that we've already sent this mail, so we simply do nothing
    - Retry updating the CRM system

# Event Sourcing

- Traditional relational database: CRUD
  - Update in place
- Change log, shadow table
- Turn it upside down: *Event journal* is the source of truth
  - Read from the event journal to create your query model
  - No more CRUD
  - Read from your event journal again: *full-text search!*
  - Read from your event journal again: *business analytics!*
- Event journal can even be a part of your API

# Eventual consistency

- Traditional approach to consistency (*transactions*)
  - Data store hides concurrent modifications of multiple areas from each other, enforcing constraints
  - Modifications typically (hopefully) fail if attempting to modify the same data
  - Even within one data store, hard to get 100% right
  - Complexity skyrockets when trying to scale beyond one data store (*distributed transactions, XA*)
- Eventual consistency
  - Embrace the flow of data through the system hitting data stores at different times
  - Embrace real time as a parameter to affect business logic
    - *Is it OK if a document I just saved doesn't show in the list until 0.5 seconds later?*
  - Apply Idempotency to all data store updates
  - Leverage Event Sourcing where possible

# Command query responsibility segregation

- CQRS: Have two separate data models (and split your API accordingly)
  - A *command* model, for API calls that only change data (and do not return data)
  - A *query* model, for API calls that only return data (and do not change data)
- Builds on CQS (Command query separation). One method can only do one of two things:
  - Perform a *command*, by having side effects (and not returning a value)
  - Perform a *query*, returning a value (and not having side effects)
- We'll see CQS again

# Character encoding history

- Early computer systems: 7 bits (last bit not transmitted, or used for parity)
- 1961: ASCII (7-bit)
- *Nøglen er sær*
- 1978: Code pages (DOS 3.3)
- 1985: ISO-8859-1
- 1991: First unicode (16-bit)
- 1996: Unicode extended to 21-bit (about 1.000.000 code points)
- 1998: UTF-8

# Line endings

- New line in text documents (including XML):
  - Windows: `\r\n` (carriage return, line feed) or in bytes: (13, 10)
  - Mac, Linux: `\n` (line feed) or in bytes: (10)
  - (old) AS400 (IBM): `0x85` (NEL, 133)
- This brings issues when collaborating
  - Both for humans and systems alike

# XML: History

Obligatory XKCD

- 1986: SGML: Standard Generalized Markup Language (first introduction of <tags></tags>, with some wild other syntaxes)
- 1990: Birth of HTML
- 1996: XML 1.0, first spec (from *text* to *data*)
- 1999: First draft for *namespaces* in XML
- 2004: XML 1.1
- 2004: XSD 1.1 (first usable definition, including namespaces)

# XML: Example

```
<?xml version="1.0" encoding="UTF-8"?>
<Invoice
 xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
 xmlns:cac="urn:oasis:names:specification:ubl:schema:xsd:CommonAggregat
 ↪  eComponents-2"
 xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicCom
 ↪  ponents-2">
 <cbc:ID>42</cbc:ID>
 <cbc:IssueDate>2004-05-24</cbc:IssueDate>
 <cac:InvoiceLine>...</cac:InvoiceLine>
 <cac:InvoiceLine>...</cac:InvoiceLine>
 <cac:InvoiceLine>...</cac:InvoiceLine>
 <cbc:ShoeSize/>
 <cac:LegalMonetaryTotal>
  <cbc:PayableAmount currencyID="USD">52.00</cbc:PayableAmount>
 </cac:LegalMonetaryTotal>
</Invoice>
```

- Extensible Markup Language
- Composes very well
  - Namespaces prevent shadowing
  - Natural order of tags can be useful
- *De facto* schema standard (XSD) has unfortunate limitations
  - Hard to express "order does not matter"
  - Hard to express "this schema can be extended with extra tags and attributes"
  - Alternatives: *schematron* (alive) and *relax-ng* (dead?)
- Still, a very sensible default choice

# JSON: History

- 2001: JSON (loading Javascript dynamically)
- 2013: First "real" JSON standard from ECMA
- 2017: Latest current JSON standard, RFC 8259

Note: No semantic versioning

# JSON: Example

```
{
  "invoice": {
    "id": "42",
    "issueDate": "2004-05-24",
    "legalMonetaryTotal": {
      "payableAmount": {
        "value": "52.00"
        "currencyID": "USD"
      }
    }
  }
}
```

- *JavaScript Object Notation*
- Started its life in the web browser (~2000)
  - XML inconvenient to deal with in Javascript back then (SAX API)
  - JSON could just be parsed as Javascript directly
- No namespaces
  - JSON is useless without context
- No (useful) types
  - JavaScript *number* is a technically a double-precision float (even though in JSON it can contain unlimited digits)
  - Even JSON schema does not remedy this
- No comments

# JSON: Parser comparison

It's just Javascript, how hard can it be

- 2018: Test of about 50 JSON parsers

Quiz:

- How many of those JSON parsers handle all edge cases correctly?
- How many of those JSON parsers handle all edge cases in the same way?

# Protobuf

```
message SearchRequest {
  required string query = 1;
  optional int32 page_number = 2;
  optional int32 result_per_page = 3;
}
```

- Very compact binary format
- Started at Google, today >70 implementations
- Built with organic versioning in mind
- Ideal for storing events of event sourcing (if you have a lot of them)

# Designing for extensibility

- Use schemes and code lists instead of fixed enumerations

```
<InvoiceAmount currencyID="USD">42.00</InvoiceAmount>
```

- Use rich data objects instead of flat numeric values
  - e.g. `Amount`, `Measurement`, `GeoCoordinate`, `Quantity`
- Use namespaces and URIs where you can

# About Spring Boot

- *Spring*: Framework providing useful abstractions for common concepts
  - Dependency injection of (mostly) singletons
  - Transaction management
  - Asynchronous messaging
  - Many, many more
- *Spring boot*: Automatically wire default singletons for various other libraries
  - Kafka
  - RabbitMQ
  - Flyway
  - JOOQ
- Sensible defaults, or magic mystery?

# Lambdas (Java 8+)

- A *lambda* is an anonymous function body
- You can write them anywhere a *Functional Interface* is expected, e.g.

```
public class LambdaDemo {
    interface MathOp {
        public int apply(int input);
    }

    static int twice(int input, MathOp op) {
        return op.apply(op.apply(input));
    }

    public static void main(String[] args) {
        int result = twice(10, i -> i + 1);

        /*
        int result = twice(10, new MathOp() {
                public int apply(int input) {
                    return input + 1;
                }
            });
         */
        System.out.println(result);
    }
}
```

# Function types (Java 8+)

- For use with lambdas, Java added a few *Functional Interfaces* in java.util.function

```
package java.util.function;

interface Function<T,R> {
    R apply(T t);
}

interface Supplier<T> {
    T get();
}

interface Consumer<T> {
    void accept(T t);
}

interface BiFunction<T,U,R> {
    R apply(T t, U u);
}
```

# Type-inferred variables (Java 11+)

- Local variables now no longer need a type
- This is especially useful with long generic types
- `var` should be your default in new code

```
import java.util.function.Function;

public class TypeInferred {

    static <T> Function<T,T> twice(Function<T,T> fun) {
        return t -> fun.apply(fun.apply(t));
    }

    public static void main(String[] args) {
        var twiceAddOne = twice((Integer i) -> i + 1);
        int result = twiceAddOne.apply(10);
        System.out.println(result);
    }
}
```

# Records (Java 14+)

```
public record Rect(float length, float width) {
    public Rect {
        System.out.println("This is a compact constructor.");
    }
}

public static void main(String[] args) {
    var r = new Rect(5, 3);
    System.out.println(r);
}
```

- A `record` is an immutable value type
  - Each argument is a private, final field
  - Getter for each argument with the same name
  - Constructor with all arguments (can have extra code as shown)
  - Appropriate `equals`, `hashCode` and `toString`
- Ideal for data classes, and perhaps domain model
- Expect JSON and XML code generators to pick these up
  - JOOQ was updated with `record` support for POJOs (for 3.15.0)

# Original java `switch` statement

```
String Animal = ???;
int legs;
switch (animal) {
    case "DOG":
    case "CAT":
        legs = 4;
        break;
    case "SPIDER":
        legs = 8;
        break;
    default:
        legs = 0;
        break;
}
```

- Is a statement (no return value)
- Only primitives, `enum` or `String`
- Can't check `null`
- Fallthrough
- Cases must be compile-time expressions (can't `case myFavouriteAnimal:`)

# Java 13: `switch` expression

```
var legs = switch(animal) {
    case "DOG", "CAT" -> 4;
    case "SPIDER" -> 8;
    default -> 0;
};
```

- No fallthrough
- Multiple values allowed
- After -> an expression is expected, which is returned

# Java 13: yield and return

```
var legs = switch(animal) {
    case "DOG", "CAT" -> 4;
    case "SPIDER" -> {
        log.debug("We got a scary animal!");
        yield 8;
    }
    default -> 0;
};
```

- You can start an expression block after ->
- Use `yield` to hand back a result from that block to the `switch` expression
- The `return` keyword will always return from the enclosing method

# Java 18: `null` allowed

We can now also use `null` in switch expressions:

```
var legs = switch(animal) {
    case "DOG", "CAT" -> 4;
    case "SPIDER" -> {
        log.debug("We got a scary animal!");
        yield 8;
    }
    case null -> 0;
    default -> 0;
};
```

# Java 18: Exhaustiveness

```
enum Animal {DOG, CAT, SPIDER}

var legs = switch(animal) {
    case DOG, CAT -> 4;
}
```

The above does not compile. We have to either:

- Add the remaining enum value

OR

- Add a `default` case

# Checking the subtype of an object

A common case is checking the subtype of a class:

```
Object o;
double result;
if (o instanceof Integer) {
    result = ((Integer) o).doubleValue();
} else if (o instanceof Float) {
    result = ((Float) o).doubleValue();
} else if (o instanceof String) {
    result = Double.parseDouble(((String) o));
}
```

# Java 16: Pattern matching for `if`

We can now introduce typed variables after `if`.

```
Object o;
double result;
if (o instanceof Integer i) {
    result = i.doubleValue();
} else if (o instanceof Float f) {
    result = f.doubleValue();
} else if (o instanceof String s) {
    result = Double.parseDouble(s);
}
```

# Java 21: Pattern matching for `switch`

Let's use the same trick with `switch`.

```
double result = switch (o) {
    case Integer i -> i.doubleValue();
    case Float f -> f.doubleValue();
    case String s -> Double.parseDouble(s);
    default -> 0d;
};
```

# Java 21: Guards

In switch expressions, we can add a predicate to any branch.

```
String result = switch(o) {
    case Integer i when i > 0 -> "+" + i;
    case Integer i -> "" + i;
    case String s -> s;
    default -> "huh?";
}
```

# Java 21: Destructuring records

```
public record Address (String city, String street, int house) {}
public record Person (String name, Address address) {}
```

We can now introduce variables for record fields, both in `if` and `switch`.

```
if (o instanceof Person (String name, Address address)) {
    System.out.println(name);
}
```

This also works for nested records.

```
if (o instanceof Person (String name, Address(String city, String
↪  street, int house))) {
    System.out.println(city);
}
```

But, we've had `var` for a while, so let's shorten this.

```
if (o instanceof Person (var name, Address(var city, var street, var
↪  house))) {
    System.out.println(city);
}
```

We can use an *unnamed pattern* to shorten this further (preview in Java 21, final in Java 24)

```
if (o instanceof Person (_, Address(var city, _, _))) {
    System.out.println(city);
}
```

# Migration management

- Relational databases change, just like code
- Change management processes in the wild
  - Schema changes just aren't allowed
  - Schema changes require a paper form, a DBA to sign off, and a 6-month process
  - Schema changes are done manually, live, on the production database, by a junior dev
- Manage your migrations: `Flyway`
  - Write incremental changes as scripts
  - Trace script execution in the database itself
  - See an example

# Interacting with data

- Several layers of abstraction
  - Direct JDBC
  - Spring `JdbcTemplate`: Thin wrapper, helps with resource clean-up
  - Spring data (*JDBC* or *JPA* variants): Automates mapping to/from Java objects
  - JOOQ: Compiles code from your database schema
- Remember databases are allowed to change?
  - Let's use the compiler to our advantage

# Introducing JOOQ

Given the following table:

```
CREATE TABLE entry (
  id SERIAL PRIMARY KEY,
  key VARCHAR(64),
  value VARCHAR(255)
);
```

JOOQ will allow you to write code like:

```
ctx.selectFrom(ENTRY).orderBy(ENTRY.ID).fetchInto(Entry.class)
```

- Type-safe table and column definitions
- Automatic mapping into generated class
- Compile errors when you remove/change columns in your schema

- Just adding the maven dependency is enough

```
<dependency>
  <groupId>org.springframework.amqp</groupId>
  <artifactId>spring-rabbit</artifactId>
</dependency>
```

- Will look for rabbitMQ locally on the default port
- Can't fail application if RabbitMQ is/goes down

# Spring Boot: Read from queue

- We use (surprise!) an annotation to read from a queue

```
@Component
public class MyComponent {
    @RabbitListener(queues = "demoQueue")
    public void handle(String message) {
        // ...
    }
}
```

- Spring will send a consumer `ack` when, and if, method returns normally
- Exceptions will cause a `nack` with redelivery (except for some specific cases)

# Spring boot: Send to queue

- Using *Publisher Confirms* is not the default (or easy) in Spring Boot
  - "Fire and forget" is the default
- Enable in configuration

```
spring.rabbitmq.publisher-confirm-type=simple
```

- Wrap your message sending code in a lambda

```
template.invoke(cb -> {
    cb.convertAndSend("demoQueue", message);

    // Potentially we can do other things in between sending the message
    ↪  and awaiting
    // its conformation.

    cb.waitForConfirmsOrDie(1000);
});
```

# Lambdas as control constructs

- Lambdas are ideal to cover large code blocks
- Let's create a simple transaction manager

```
class Transactional {
    public <T> T inTransaction(Function<Connection,T> fun) {
        var conn = DriverManager.getConnection(url);
        conn.setAutoCommit(false);
        try {
            fun.apply(conn);
            conn.commit();
        } catch (Exception x) {
            conn.rollback();
        } finally {
            conn.close();
        }
    }
}
```

# Lambdas as control constructs (contd.)

- Now we can wrap a lambda and get a transaction around the block

```
void updateUser(String name, String pet) {
    inTransaction(conn -> {
        validatePetName(conn, pet);

        var stmt = conn.prepareStatement("UPDATE users SET name=?,
        ↪  pet=?");
        stmt.setString(1, name);
        stmt.setString(2, pet);
        stmt.executeUpdate();
        stmt.close();
    });
}
```

# Pure functions

- Remember CQS (Command query separation)
- A *Pure function* will
  - Given the same arguments, will always return the same value
  - Not have any side effects (I/O, global state)
- Pure functions are great!
  - Trivial to test
  - Compose really well
  - Easy to reason about

# Immutability

- Let's have a look at this pure function

```
int calculateAverage(List<Integer> items) {
    // ...
}
```

Is this a pure function?

# Immutability (cont.)

- Well, here's an implementation

```
int calculateAverage(List<Integer> items) {
    items.clear();
    items.add(42);
    return 42;
}
```

- To reason about pure functions, we need immutable data structures
  - Lists that can not be modified
  - Maps as well, and many others
- We still want to create modified copies (of course)

# I don't need immutability!

- There are many other good use cases for immutable data structures
  - Caches
  - Callback APIs
  - Parallel processing
  - Messaging



- Library for Java 8+ for functional programming and immutability
- As an example, let's look at io.vavr.collection.List (a linked list)

```
import io.vavr.collection.*;

List<Integer> list1 = List.of(1, 2, 3);

// Create a new list with more values:
List<Integer> larger = list1.append(4).append(5);

// Let's turn all integers to strings
List<String> strings = larger.map(i => i.toString());

// Let's turn each list into several elements and flatten them:
List<Integer> replicated = list1.flatMap(i -> List.range(0, i));
// 0, 0, 1, 0, 1, 2
```

# VAVR (contd.)

- Other useful types from `io.vavr.collection`:
  - `Vector<T>`; (an immutable *hash trie*, with efficient inserting and removing)
  - `HashMap<K,V>`; (an immutable hash map)
- Useful control data structures in `io.vavr.control`:
  - `Option<T>` : single-valued element, which can either be `None` or `Some(value)`
    - This is the preferred way of handling potentially-absent values
  - `Either<L,R>` : single-valued element, which can either be `Left(leftValue)` or `Right(rightValue)`
    - This is the preferred way of handling values (R) that may have failed with an error (L)
  - `Validation<E,T>` : variant that can combine multiple errors, or a result value

# Null-free style

- `NullPointerException` is hiding in every `.` in traditional java
- There's no need to use `null` anymore to carry semantic value in new applications
  - In plain JDK, there's `java.util.Optional`
  - For more integration with collections, there's VAVR's `Option`
- `@NotNull` ?
- Simple rule: the word `null` must not occur in your source code
  - Exception: integration with libraries that expect `null`
- `Option` (or `Optional`) is fine both for method arguments, return types, and fields
  - However, consider method overloading as well

# Composing Spring annotations

- Let's listen from a message on RabbitMQ

```
@RabbitListener(queues = "demoQueue")
@Transactional
@Timed("demoQueue.message")
public void handleMessageFromRabbit(String message) {
    System.out.println("We've got a message!");
}
```

- Well, we can see that Spring sprinkles magic on this method which
  - Invokes it when a message arrives on RabbitMQ
  - Wraps it in a database transaction
  - Measures how long it takes
- But, in what order?
- And, what code implements these concerns?

# Composing lambdas

- Let's look at this alternative method implementation

```
@RabbitListener(queues = "demoQueue")
public void handleMessageFromRabbit(String message) {
    transactions.doInTransaction(ctx -> {
        myCustomMetric.measureTime(-> {
            System.out.println("We've got a message!");
        });
    });
}
```

- We keep the `@RabbitListener` (for now, ask us about streams!)
- Other concerns' ordering is trivial to follow
- Implementation is trivial to find

# Functional style

- Java libraries are starting to use lambdas as defaults over annotations
  - Resilience4J uses lambdas to wrap circuit breaker, bulkhead
  - JOOQ can use lambdas to demarcate transactions
- Advantages
  - Discoverability, Composability, Testability
- Sneak peak into lambdas for handling HTTP (pekko-http):

```
Route route = path("users", () ->
    concat(
        get(() ->
            complete("Here could be a list of users!")
        ),
        post(() ->
            entity(Jackson.unmarshaller(User.class), user ->
                onSuccess(addUser(job), r -> complete("User added!"))
            )
        )
    )
);
```

# API guide

- Follow GUIDE.org
- Browse the code, if time

# Let's build a chat room

- Extend `demo-project` with a new business domain: *chat rooms*
- Create the following new APIs:
  - Post a message to a room (creating the room if it doesn't exist)

```
POST http://localhost:8080/rooms/myroom/messages
Content-Type: text/plain

Here's a chat message going to room "myroom"!
```

- Get the current messages in a room

```
GET http://localhost:8080/rooms/myroom/messages
Accept: text/plain
```

Data need not be persisted (yet).

# Bonus points:

- Create a data schema migration and actually store the messages
- Have an API to list rooms
- Tie messages to users instead of anonymously

# Discussion

- Let's see what everyone has come up with

# Three alternatives

- Group into teams
- Pick the alternative that most suits you (or which you like best)
  - Alternative A: Replace your monolith
  - Alternative B: Extend an existing microservice
  - Alternative C: Write some code for the demo project

# Part one: Describe monolith

Target: individual developers, or developers that have worked on the same monolith.
Groups of 1-4

- Pick a recent project where you have worked on a *monolith*. The bigger the better.
- Create a (rough) sketch that depicts the monolith the most important other systems it communicates with
  - Include both clients and dependencies
  - Include data stores and queues
  - Include cloud-based services
  - For each interface, describe (generally) which protocol or format is used, how often it's used, and the size of messages
- Create a (rough) sketch that depicts the monolith, its internal structure, and the team(s) changing those parts
  - Try to include the size of teams and how often they perform changes

- Using the techniques from this course, draw a set of candidate microservices that can take over part(s) of the monolith
  - Which business domains
  - What APIs
  - Which data stores

- Target: individual developer, or developers that have been on a (small) service together.

- Create a (rough) sketch that depicts the service the most important other systems it communicates with
  - Include both clients and dependencies
  - Include data stores and queues
  - Include cloud-based services
  - For each interface, describe (generally) which protocol or format is used, how often it's used, and the size of messages
- Create a (rough) sketch that depicts the service, its internal structure, and the team(s) changing those parts
  - Try to include the size of teams and how often they perform changes

- What do we change in our architecture if we expect (separately):
  - 100x as many concurrent users
  - 1000x the size (in bytes) of incoming requests
  - 10.000x the amount of stored data

# Finish the chat

Suggestions:

- Create a data schema migration and actually store the messages
- Have an API to list rooms
- Tie messages to users instead of anonymously
- Add a bulkhead around some APIs
- Remove single points of failure
- Typing notifications

# Temperature check

| Name | @@@@@ I'm learning new stuff | @@@@@ I know microservices |
| --- | --- | --- |
| | | |

# Schedule, Day 2

| Time | Duration | Activity | Weight |
|---|---|---|---|
| 09:00 | 00:10 | Welcome, Outline/Agenda | |
| 09:10 | 00:10 | Recap of day 1 | |
| 09:20 | 00:45 | REST patterns | 24 |
| 10:05 | 00:20 | Getting your service used | 10 |
| 10:25 | 00:10 | REST examples | 24 |
| 10:35 | 00:60 | Breakout 4: Event storming | |
| 11:35 | 00:30 | Lunch | |
| 12:05 | 00:30 | Breakout 5: API design | |
| 12:35 | 00:25 | Discussions of REST results | |
| 13:00 | 00:20 | Microservice life cycle | 12 |
| 13:20 | 00:10 | Security architecture | 5 |
| 13:30 | 00:15 | Strategy and team dynamics | 4 |
| 13:45 | 00:40 | Asynchronous Java | 22 |
| 14:25 | 00:55 | Breakout 5: Streaming crawler | |
| 15:20 | 00:30 | Presentations of async Java | |
| 15:50 | 00:10 | Wrap-up, reserved time for extra subjects | |

# Recap of day 1

Who can tell us something about:

lamdas

monitoring

partitioning

sharding

bulkhead

circuit breaker

resilience

tracing

alerting

null-free

# Homework presentations

- Share your services (and/or code) with us!

# Break

- Make sure to get the latest project changes:
  - `git pull` to get latest updates

- World-wide web (1990): HTTP over internet, with hypermedia (HTTP)
  - Unprecedented scaling
  - Applications (e.g. Facebook, Amazon) can develop continuously without clients (browsers) breaking
    - *(at least, until they figured out native clients means no ad-blockers. . . )*
  - Managed to survive 20+ years in a wild changing landscape, with limited technical debt
    - Most of HTTP and HTML are still relevant
- Apparently, it's possible to perform heterogeneous systems integration without any
  - legal contracts,
  - deep specifications, or
  - personal knowledge
- Try pulling that off in your enterprise!
- Apply the WWW success for system-to-system communication
  - `RE` presentational `S` tate `T` ransfer
- Request-based from *client* to *server*
  - Distinctly separated roles that two systems or actors play when handling a request
- Stateless
  - Request contains all information needed to process it (instead of, e.g. the TCP connection socket)
- Caching
  - Responses must clearly state, and have sensible defaults, on how content can be cached
- Uniform interface
  - All components are accessed the same way
- Layered system

- Intermediaries can be transparently inserted between client and server (load balancers, proxies, security gateways, . . . )

In REST, the *client* accesses a *resource* on the *server*, through a *request*.
A resource:

- Is a noun, e.g. *user*, *invoice*, *setting*, but also *transaction*, *order status*, or *deletion process*
- Can have several representations, e.g. XML, JSON, HTML, picture, small, large
- Is accessed through one or several URLs
  - `/users/15`, `/users/latest`, `/users?name=Santa` might all return the same resource
- Is interacted with through a limited set of verbs (more on that later)

Remember your event storming workshop?

- Text-based protocol over TCP
  - Client sends a request (with *verb*, *headers*, and optional *body*)
  - Server sends a response (with *status line*, *headers*, and optional *body*)
  - (since HTTP 1.1) Client sends a new request, etc.

Client sends:

```
GET /cats/latest?fur=white&size=small
Accept: image/png
User-Agent: Mozilla/5.0
```

Response then comes in:

```
200 OK
Content-Type: image/png
Content-Length: 53748

[...kitten goes here...]
```

# HTTP Verbs

- Predefined verbs imply important caching and retry semantics

| Verb | Safe to retry? | Idempotent? | Request body? | Response body? | Cache resp |
|---|---|---|---|---|---|
| GET | yes | yes | - | yes | yes |
| HEAD | yes | yes | - | - | yes |
| PUT | - | yes | yes | - | - |
| POST | - | - | sometimes | sometimes | - |
| DELETE | - | yes | - | - | - |

- Why wouldn't `PUT` or `DELETE` be safe to retry?
- Rest is not RPC

# Example HTTP status codes

The *status line* contains a code and then a short description. The description is not predescribed, and sometimes contains useful information.

- `200 OK`
  - The request succeeded. Typically a response body is present.
- `201 Created`
  - The request succeeded, and a new resource was created as a result.
- `204 No Content`
  - The request succeeded, but no content is available.
- `302 Found`
  - The resource was found at a different URL, which is returned in the `Location` header.
- `404 Not Found`
  - The resource does not exist. This does *not* necessarily mean that an API endpoint does not exist.

This is not a full list. See the HTTP official status codes or a more graphically-accessible variant.

# Example HTTP Headers

- `Accept: image/*`
  - Sent in a *request* to indicate the MIME types that the client prefers for this request (but there's no guarantee)
- `Content-type: image/png`
  - Sent in a *request* or *response* to indicate the actual MIME type of the body
- `Content-length: 5124`
  - Sent in a *request* or *response* to indicate the size of the body in bytes (if known)
- `Last-Modified: Wed, 21 Oct 2015 07:28:00 GMT`
  - Sent in a *response* to indicate when that resource was last changed

This is not a full list.

- Find resources for your domain
  - Perhaps using an *Event Storming* workshop (from *Domain-Driven Design*)
- Use CQRS (Command Query Response Segregation)
  - Find representations for those resources (current state and/or events): `GET`
  - Find commands affecting those resources (creation, modification, transactions): `POST`, `PUT`, `PATCH`, `DELETE`
- Size limits on everything (do we need to stream or read it in memory?)
- XML, JSON, CSV, text, protobuf (more content-type negotiation later)
- Decide on a *Service Level Objective* for your API (yes, already now)

# An API is an interface

- *Application Programming Interface*
  - It's how external components affect what our service does
  - Better lay down some rules
- But our service is only used by our team, we don't need documentation!
- Ideal for test-first development
- Where do I put my private API?

# Example API

- Semantic format for describing REST APIs
  - RAML: YAML-based, better re-use, easier to write by hand
    - Content format agnostic
    - Unfortunately, not that maintained anymore
  - OpenAPI: JSON/YAML-based, more popular
    - JSON-biased
    - "endpoint"-biased
- Let's look at an example API example API together
  - Its RAML source is available

# Content-type negotiation

- Embrace content-type negotiation (XML *and* JSON, not XML *or* JSON)
- XML API:
  - Do create XSD for your data types, but communicate how it should be interpreted
  - Do you reserve the right to add new tags and attributes?
- JSON API:
  - Create JSON schemas for everything
  - In addition, verbosely describe all numeric types and their intended usage

# But I've written the documentation!

- Just a list of endpoints may not be enough for some developers
- Lot of context and assumed knowledge
  - Ubiquitous language may not extend to all new API users
  - Lack of experience with JSON, XML, HTTP headers

# Different people, different learning styles

- Write a developer guide that describes typical scenarios from a user's perspective
  - How to get started (e.g. get an SSL certificate)
  - How to list widgets in XML or JSON
  - How to create a new widget
- There's no shame in taking an English technical writing course
- Pick tooling that suits your way of working (e.g. `HTTPie`, `org-mode` with `org-babel`, ...)

# Priorities!

- What's the first thing you do when you get to your office?
- Users will be curious about your service status
  - If your users are internal, give them access to the actual dashboard
  - In fact, consider giving them access to your source code and issue tracker as well

# Designing your dashboard

- Your dashboard should be showing
  - System metrics (load average, disk space, CPU usage, memory usage, network I/O, disk I/O)
  - Your process' metrics (CPU usage, memory usage)
  - Your VM's metrics (Heap committed, heap used, GC time, thread count, log count)
  - Your framework's metrics (HTTP server open connections, HTTP client open connections, response times, response errors)
  - Your business metrics (number of pets signed up, total invoice amount, size of received chat messages)
- For each environment, after a few days examine the graphs
  - Establish a baseline, and create an alert for *each* metric

# Resources have versions

- Servers can include an `ETag`, which specifies which *version* of a resource is being served

```
GET http://example.com/widgets/15

200 OK
Content-Type: application/json
ETag: "524"
```

- No guarantees are made about the content of `ETag`, but often APIs will document what it represents, e.g.
  - A timestamp of some sort
  - A monotonically-increasing number
  - A hash of the latest content

# Conditionally retrieving a resource

- If the latest `ETag` we have seen is "524", we can poll for changes
- The `If-None-Match` header will *only* execute our request if the `ETag` has changed

```
GET http://example.com/widgets/15
If-None-Match: 524

304 Not Modified
```

- The server will not send any response if the resource is still at this version

# Optimistic offline lock

- The `ETag` is also useful to make sure nobody else has edited a resource that we're writing back
- The `If-Match` header will *only* execute our request if the `ETag` matches

```
PUT http://example.com/widgets/15
If-Match: 12345
Content-Type: application/json

{ /* ... some content ..* }

412 Precondition Failed
```

# Resources can be modified

- Servers can include a `Last-Modified` tag, which specifies *when* a resource was last changed
- This can be useful in addition to an `ETag` tag

```
GET http://example.com/widgets/15

200 OK
Content-Type: application/json
ETag: "524"
Last-Modified: Wed, 21 Oct 2015 07:28:00 GMT
```

- Request header exist that perform checks against the last-modified date, like `ETag`:
  - `If-Modified-Since` executes the request *only* if the last-modified is past the given date
  - `If-Unmodified-Since` executes the request *only* if the last-modified is at most the given date

# Preventing caching

- For service-to-service REST calls, we generally don't worry about caching
- For web browsers, we often want to disallow caching of REST responses
  - Include `Cache-Control: no-cache`

# Resource representation

- The same REST URI is allowed to have several representations
  - XML, JSON or Protobuf
  - Short or long
  - Version 1 or version 2

# Specifying resource representation

- The server specifies the representation of a resource
  - The `Content-Type` resource header
- This is typically a well-known value
  - `text/xml`
  - `application/json`
  - `application/protobuf`
- But it doesn't have to be
  - `application/vnd.example.myresource.v1+json`
  - `application/vnd.example.myresource.v2+json`
  - `application/vnd.example.myresource.short+json`
  - `application/vnd.example.myresource.long+json`

# Requesting a resource type

- The client sends an `Accept` header with the representations it wants/understands
- In case of a single representation:

```
GET http://localhost/myresource
Accept: application/json
```

- In case multiple representations are alright (order has no semantic meaning):

```
GET http://localhost/myresource
Accept: application/json, text/xml
```

- Multiple representations are alright, but preference for xml:

```
GET http://localhost/myresource
Accept: application/json;q=0.9, text/xml
```

# Serving resource alternatives

- Content-type negotiation is complex to implement
- How easy it is to support depends on your framework
  - *Spring Boot* has many different ways to manage resource representation
    - Look into `HttpMessageConverter`, so you can take control
  - Others, e.g. `pekko-http` has a marshaling infrastructure that directly models content-type negotiation

# Case: REST API to represent workflow instances

- Start a new workflow
- See which human is working on the case
- Quickly resume if system is working on the case

# REST is about resources

- For slow-running processes, make the process itself a resource, e.g.
  - `/workflows/`
  - `/transactions/`
  - `/cases/`
- You can now reason about individual processes
  - Query state, affect them, delete them, see changes

# Observing change on one resource

- Tell client to periodically poll
  - Use `If-None-Match` for early exit
  - Use heavy caching on the server-side to reply to polls as early as possible

# Observing change on a set of resources

- Build your system using *Event Sourcing*
- Expose your event journal (or a light, or filtered version) as a REST resource
  - This can be done regardless of storage (JDBC, Cassandra, Kafka)
- Various candidates for the data format
  - Plain

```
GET http://localhost/journal/events?since=Wed+May+26+11:59:05+2021+UTC&
↪  limit=50
Accept: application/json
```

  - Hanging GET

```
GET http://localhost/journal/events?since=Wed+May+26+11:59:05+2021+UTC&
↪  limit=50&maxwait=60000
Accept: application/json
```

  - Server-sent events (SSE)

```
GET http://localhost/journal/events?since=Wed+May+26+11:59:05+2021+UTC
Accept: text/event-stream
```

  - Web sockets

```
GET http://localhost/journal/events?since=Wed+May+26+11:59:05+2021+UTC
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: HSmrc0sM1YUkAGmm5OPpG2HaGWk=
Sec-WebSocket-Protocol: chat
```

# Semantic versioning in REST

- Often used for library dependencies and packaged software releases
- Version number has three parts (*major*, *minor*, *patch*): version `2.0.15`
  - A new release always must have a new version
  - If a release has no new functionality (only bugfixes), increase the *patch*
  - If a release has new functionality that doesn't break API promises, increase the *minor*
  - If a release has new breaking functionality, increase the *major*
- How does this relate to REST?

# Semantic versioning in REST (cont.)

- How does this relate to REST?
  - It doesn't!
  - REST is a call to a remote system
    - Could be deploying new versions multiple times per day
  - The whole point is the client *doesn't* want (or need) to see those
- OK, what do we do instead?
  - Version across all HTTP dimensions

# Versioning in body structure

- Many extensions fit fine into existing body structure
  - Adding of fields
  - Adding of values to enumerations or code lists
- If DDD has done its work, terminology should mostly hold

# Versioning in content type

- If a breaking change is needed
- It might be limited to only one content type
- Client requests old version:

```
GET http://localhost/myresource
Accept: application/vnd.example.myresource.v1+json
```

- Client requests new version:

```
GET http://localhost/myresource
Accept: application/vnd.example.myresource.v2+json
```

# Versioning in query parameters

- Don't do this
  - Query parameters affect *which* and *what* resource(s) are returned, not *how*
- The meaning of query parameters may themselves be versioned

# Versioning in path

```
GET http://localhost/service/versions/1/myresource
```

- Often used as first choice
- Should be your last resort:
  - Your path is the name of your resource
  - Your DDD workshop (probably) didn't event storm about "versions"
  - Your system (probably) doesn't have 2 complete implementations
  - This does often not reflect reality

# Versioning using custom headers

- Client sends a custom header of the API version they've implemented against
- Server sends a custom header of the API version that's current
- This does kinda work
- Fairly weak way to work around *actually* dealing with semantic changes and compatibility

```
GET https://api.github.com/search/issues?q=windows+label:bug+language:p
↪  ython+state:open&sort=created&order=asc
Accept: application/vnd.github.text-match+json

200 OK
Content-Type: application/vnd.github.text-match+json
{
  "text_matches": [
    {
      "object_url":
      ↪  "https://api.github.com/repositories/215335/issues/132",
      "object_type": "Issue",
      "property": "body",
      "fragment": "comprehensive windows [...] ter.\n",
      "matches": [ ... ]
    }, [...]
  ]
}
```

Notes:

- Using a custom content-type to indicate a special flavor of JSON
- Relying on `GET` to indicate a read request

# AWS: Create EC2 instance

```
GET https://ec2.amazonaws.com/?Action=RunInstances&ImageId=ami-2bb65342
↪  &MaxCount=3&MinCount=1&Placement.AvailabilityZone=us-east-1a&Monito
↪  ring.Enabled=true&Version=2016-11-15&X-Amz-Algorithm=AWS4-HMAC-SHA2
↪  56&X-Amz-Credential=AKIDEXAMPLE%2F20130813%2Fus-east-1%2Fec2%2Faws4
↪  _request&X-Amz-Date=20130813T150206Z&X-Amz-SignedHeaders=content-ty
↪  pe%3Bhost%3Bx-amz-date&X-Amz-Signature=525d1a96c69b5549dd78dbbec8ef
↪  e264102288b83ba87b7d58d4b76b71f59fd2

200 OK
[... EC2 server is started...]
```

# AWS: Create EC2 instance

Notes:

- a `GET` verb is used to have side effects!
- No resource representation of the actual server to be created
- Proprietary authentication mechanism, and using the URL for this

# Flicker's image search

```
GET http://api.flickr.com/services/rest/?method=flickr.photos.search&ap
↪  i_key=xxx&text=trysil&format=rest&auth_token=xxx&api_sig=xxx
Accept: text/xml

200 OK
Content-Type: text/xml

<?xml version="1.0" encoding="utf-8" ?>
<rsp stat="ok">
    <photos page="1" pages="121" perpage="100" total="12050">
        <photo id="12185296515" owner="110367434@N02"
        ↪  secret="7bf83bc507" server="3714" farm="4" title="#wall
        ↪  #clock #wood #old #Norway #Trysil #travel #travelling"
        ↪  ispublic="1" isfriend="0" isfamily="0" />
        <photo id="12185880206" owner="110367434@N02"
        ↪  secret="c8042c1764" server="7382" farm="8" title="Good
        ↪  morning!   #Norway #Trysil #window #snow #beautiful
        ↪  #landscape #travel #travelling #polar #expedition"
        ↪  ispublic="1" isfriend="0" isfamily="0" />
        ...
        <photo id="11793639173" owner="40644602@N08" secret="ba2bdabf5c"
        ↪  server="7451" farm="8" title="by beateorten
        ↪  http://ift.tt/1dHDdQL" ispublic="1" isfriend="0"
        ↪  isfamily="0" />
    </photos>
</rsp>
```

# Flicker's image search

Notes:

- Overloading of methods in the request URL
- Overloading of content type in the request URL
- Overloading of HTTP status code in the response
- No obvious way to explore the API further (how do I load a photo?)

# Domain scope

- Let's find a domain and scope for the events we want to discover
  - Is there a shared system, or domain, most of you have worked on?
  - Is there a shared system, or domain, most of you know is important for your business?

# Alternative: Pet Shop

(skip if domain is found)

- Let's model a pet shop!
  - Our family owns a pet shop, which has a building that houses pets for sale
  - We regularly sell pets, and re-stock
  - Pets need to be fed
  - Some pets have special other needs
  - We only want to house cute pets!
- We want to automate as much as we can, and hence hold an event storming workshop

# Big picture

- Distribute orange post-its
- Remember, an event is in PAST TENSE, e.g.: *Missiles have been launched User has subscribed to newsletter*
- Designate a wall as space
  - Create time marker. Time flows roughly from left to right (where relevant)
- Start with "pivot" event in center
- Write other events that come to mind
  - Order with existing events, keeping time
  - Feel free to rename as discussions occur

# Big picture: Need more?

- Full narrative (1 person)
- Reverse narrative

# Pivotal events and boundaries

- *Pivotal event* is an event that is of particular business importance (and value)
  - Spawns new business processes
  - Involves new stakeholders
  - Commits the business to a financial risk or responsibility
  - Confirms the business receiving a financial benefit
- Can we, along pivotal events, distinguish *Bounded Contexts* in our events?

# Process modeling

- Distribute blue, yellow, green and pink post-its
  - Blue: *command*
  - Yellow (small): *actor (persona)*
  - Pink: *external system, department, time*
  - Green: *read model*
- Remember, a command is in IMPERATIVE, e.g. *Launch missiles Register user subscription request*
- Consider limiting scope to a part of what we modeled as Big Picture
- Select important events, that related to something a system could do for us
  - What command could cause this event? (blue)
  - Who or what can trigger this command?
    - Who: Actor (yellow)
    - What: External system (pink)
  - What information is needed to construct the command (green)

# Aggregates

- What nouns have we discovered that are good candidates for aggregates?
  - Yellow (large): *aggregate*
  - Group the commands by aggregate
- What aggregates would be good candidates for microservices?

# Link to BDD

- Take another look at some sets of related events and commands
- Can you formulate a sentence:
  - *Given* (a statement related to events happened before)
  - *When* (an event occurs, or a command is sent)
  - *Then* (another event occurs, or another command is sent)

# Design the API of the system we've created events for

- Identify *resources* from *aggregates* (and other nouns on the events)
- The *commands* are good candidates for resource changes (`PUT`, `POST`, `DELETE`)
- The *read model* notes are a good starting point for `GET`
- Start with resources (paths, query parameters)
- Then, add http methods
- Add request and response body examples if you have time
- View `demo-project/documentation/demo-api.raml` if you need an example RAML file
- Hints:
  - RAML spec: `https://github.com/raml-org/raml-spec/blob/master/versions/raml-10/raml-10.md/`
  - RAML documentation: `https://raml.org/`
  - OpenAPI spec: `https://spec.openapis.org/oas/v3.2.1.html`
  - OpenAPI online editor: `https://editor.swagger.io/`

# Alternative: Design the API for an invoicing system

- We're writing an invoicing service, that can send and receive invoices between companies
- We need APIs to:
  - Send an invoice
  - Read back previously-sent invoices
  - Check if we have received any invoices, and
  - Read received invoices
- You don't have to design an invoice format
  - You can assume they're plain text or PDF, if you want
- View `demo-project/documentation/demo-api.raml` if you need an example RAML file
- Hints:
  - RAML spec: https://github.com/raml-org/raml-spec/blob/master/versions/raml-10/raml-10.md/
  - RAML documentation: https://raml.org/
  - OpenAPI spec: https://spec.openapis.org/oas/v3.2.1.html
  - OpenAPI online editor: https://editor.swagger.io/

# Developing a new service

- I want to write a new micro service!
  - I need a database, a queue, the filesystem for some caching
  - Oh, and I'm talking to twitters API, and our home-grown analytics API
- How do I deal with these dependencies during day-to-day development?
  - "Leaf" dependencies: often OK to run directly (e.g. data stores)
  - "Node" dependencies (other microservices): often have dependencies of their own
    - You know its API, right?
    - Mock it! Wiremock, or any simple http server

# Running dependencies

- Maintain a `docker-compose` file for your project
  - Real dependencies: they're probably on `docker-hub` already
  - Mocks: use the `build` feature if needed
- New developers can get started instantly

# Developing a new feature

- Don't hide your new feature on a branch
- Release early and often
  - But only activate it in certain environments and/or users
- Feature flag
- A/B testing

# Unit tests OK, Integration tests not

graphics/tests.mp4

# Introducing bugs

- Rate of bugs introduced into systems are a function of
  - Developer experience
  - Development environment (physical and technological)
  - Methodology
- Evaluate changes in each of these if you want to increase quality

# Finding bugs

- Fixing bugs is more expensive, the later they are found
  - While writing code: just think of different solution
  - While code is in review: communication, context switch, and the above
  - While code is in user testing: (much) more communication, context switch, and all the above
  - After code is released: (even) more communication, impact analysis in data, and all the above

source: testomat/IBM and source: forbes

# Preventing bugs

- Test at different layers
  - On code itself: Pair programming
  - On one unit (e.g. class): *Unit tests*. Run in seconds.
  - On one service (e.g. rest API): *Component tests*. Run in tens of seconds.
  - On a suite of services (e.g. UI): *End-to-end tests*. Run in minutes.
  - On your entire infrastructure: *Smoke tests*. Run periodically, on production, with external dependencies
- All of the above must be part of your process and fully automated

# Getting your service out there

"*All software has a test environment. Some software is lucky to have a separate production environment as well.*"

- unknown

# Doing deployments

- Automate the environments themselves (`helm`, `kustomize`, `terraform`, `vagrant`, ...)
- All deployments to all environments must be automated
- It's OK to have gatekeepers, e.g.
  - After a PR is merged, automatic deploys are done to `dev` and `staging` environments
  - The `prod` environment requires a manual button press
- Forward deploy only
  - Rollbacks are a pain
  - Your next deploy is only minutes away
  - Emergencies should be rare (testing, early release, multiple environments)
- Or, semantic versioning

# User-to-service authentication

- I want code running on a user's computer to call me (let's assume web browser)
- OpenID Connect, simplified flow:
  1. *Resource owner* wants *client* to log on to *authorization server*
  2. Client is redirected to authorization server
  3. User verifies trust of authorization server and logs on
  4. Authorization server redirects client back (with authorization code)
  5. Client contacts resource owner with *code*
  6. Resource owner exchanges code for *token*
  7. Token can be used in `Authorization: Bearer` http header

# Service-to-service authentication

- I want code running on other backend services to call me (outside of the context of a user)
- Mutual TLS
  - Server has a certificate, proving it's who it claims
    - Client has established trust on a root certificate, having signed the server certificate
  - Client has a certificate, proving it's who it claims
    - Server has established trust on a root certificate, having signed the client certificate
- In practice
  - Create (or purchase) a root certificate for your business, lock it tight
  - Create intermediate CAs for particular roles, e.g. for singing micro-services
  - Use *Certificate Signing Requests* to reflect real business flow
  - For your service clients
    - Have server sign client certificates directly
    - Or, delegate to an intermediate CA, and implement whitelisting
- Confirm that OCSP (*Online Certificate Status Protocol*) can be used to revoke certificates

# Authorization checks

- Prefer to keep internal to service
- Replicate user memberships through event sourcing
- Synchronous calls least favorable choice

# Microservices and agile

- One service is written, maintained, deployed by one team
  - Including its data
- Supporting stakeholders
  - How is our service found?
  - How are our dashboards discovered?
  - How is our team contacted?
- Embrace change
- Team(s) in same time zone as stakeholders (which includes users)
  - Distributed users? distributed team!
- Consider Data Mesh architecture
  - You are responsible for others being able to access data
  - In a way that is useful for their use cases

# Migrating your monolith

We want to finally get rid of our monolith!

- Chainsaw anti-pattern
  - Don't try to haphazardly cut your application in two
- Strangler pattern
  - Find a project that updates / replaces a small part of your application
  - Take the opportunity to write that as a microservice
  - Update your monolith to call your new new microservice
  - Rinse and repeat
- Quick win: modularization
  - Maven modules
  - Static package dependency analysis

# Do we need a separate dev/ops team? (no)

- Automate everything (rolling production deploy)
- Deploy in the morning, monitor your dashboards
- However, "infra tooling" or "platform" team can be helpful
- The same holds for the "DBA" team

# Handling concurrency

- We expect very large, slow calls (but few of them)
  - e.g. a slow upload through a mobile network
  - Use `InputStream` and just slowly read (or `OutputStream` and slowly write)
  - Block a thread per request
- We expect many simultaneous requests, but small ones
  - e.g. distributing events to a great many consumers
  - Use asynchronous servlets (callbacks) or asyncronous Spring MVC (`CompletableFuture`)
  - Request thread is freed (while typically another does the work)
- We expect many simultaneous requests, and they're big and slow
  - Now what?

# A bit of perspective

Latency comparison numbers

| Activity | Nanoseconds | |
|---|---|---|
| L1 cache reference | 0.5 ns | |
| Branch mispredict | 5 ns | |
| L2 cache reference | 7 ns | |
| Mutex lock/unlock | 25 ns | |
| Main memory reference | 100 ns | |
| Compress 1K bytes with Zippy | 3,000 ns | |
| Context switch | 10,000 ns | (if working set in L2) |
| Send 1K bytes over 1 Gbps network | 10,000 ns | |
| Context switch | >100,000 ns | (bigger working set) |
| Create a new thread in Java | 100,000 ns | |
| Read 4K randomly from SSD* | 150,000 ns | |
| Read 1 MB sequentially from memory | 250,000 ns | |
| Round trip within same datacenter | 500,000 ns | |
| Read 1 MB sequentially from SSD* | 1,000,000 ns | |
| Disk seek | 10,000,000 ns | |
| Read 1 MB sequentially from disk | 20,000,000 ns | |
| Send packet CA->Netherlands->CA | 150,000,000 ns | |

# Threads are expensive

- Creating a new thread takes a long time
- Each thread eats >1MB of overhead (stack, housekeeping, ...)
- Scheduler has no insight to how *busy* a thread is
- Keep working on the same data for as long as possible

# Don't call us, we'll call you

- Easy to grasp and understand

```
void makeHttpRequest(String url, Consumer<String> handleBody) {
    System.out.println("Pretending a request to " + url);
    handleBody.accept("Here's your body!");
}

makeHttpRequest("http://test", body -> {
    for (int i = 0; i < 1; i++) {
        System.out.println("Got a results: " + body);
    };
});
```

# A bit too simple?

- No guarantees which thread will invoke the callback
- Up the the implementer to handle locks
  - Did we talk about deadlock yet?
- Callback "hell"

```
makeHttpRequest("http://test", body -> {
    writeDatabase(new Entry(body), onDone -> {
        sendRabbitMQMessage("We're getting there!", onAck -> {
            log.debug("Finally.");
        });
    });
});
```

# History

- `Future` introduced to Java in Java 7
  - Handle to an *asynchronous* computation in progress (typically on a different thread)
  - Basically can only `.get()`, blocking the calling thread
  - Cancelling using `.cancel()` usually does nothing
- `CompletableFuture` added in Java 8
  - Extends future with monadic operators
  - Although curious API choices: `.thenApply()` vs `.map()`, `thenCompose()` vs `.flatMap()`

# CompletableFuture examples

Let's say we have a slow background computation

```
CompletableFuture<Integer> compute(int input) {
    return CompletableFuture.supplyAsync(() -> {
        System.out.println("Starting computation for " + input);
        try { Thread.sleep(200); } catch (InterruptedException x) {}
        return input * input;
    });
}
```

We can turn it into a string once the computation completes:

```
CompletableFuture<String> willbe = compute(4).thenApply(i ->
↪   i.toString());
System.out.println("Printing now!");
System.out.println(willbe.get());
```

# CompletableFuture examples

We can also wait on another computation, when the first one completes:

```
CompletableFuture<Integer> composed = compute(4).thenCompose(i ->
↪  compute(i));
System.out.println(composed.get());
```

# Summary

- `CompletableFuture` is more expressive than callbacks
- Wacky API names
- Complicated error handling
  - Exceptions wrapped in `CompletionException`, because of cancellation (that doesn't work)

# VAVR's Future and Promise

- VAVR includes a `io.vavr.concurrent.Future` with a nicer design:
  - `map` and `flatMap` to compose (just like `Option` and `Seq`)
  - proper division of roles
    - `Future` holds a reference to an ongoing calculation
    - `Promise` holds a value that can be eventually set

```
Future<Integer> compute(input) {
    var result = Promise.<Integer>make();
    launchBackgroundProcess(() -> {
        // invoked when process completes
        result.succeed(42);
    });
    return result.future();
}
```

# Wrapping up

- Howeer:
  - Does this help with handling our big, slow, input, with many concurrent users?

# Reactive manifesto

- Developers realized that "one thread per request" and "exceptions" wasn't cutting it anymore
- 2016: The Reactive Manifesto (30k+ people). Reactive systems are:
  - *Responsive*
    - The system responds in a timely manner if at all possible.
  - *Resilient*
    - The system stays responsive in the face of failure.
  - *Elastic*
    - The system stays responsive under varying workload.
  - *Message Driven*
    - Reactive Systems rely on asynchronous message-passing to establish a boundary between components that ensures loose coupling, isolation and location transparency.

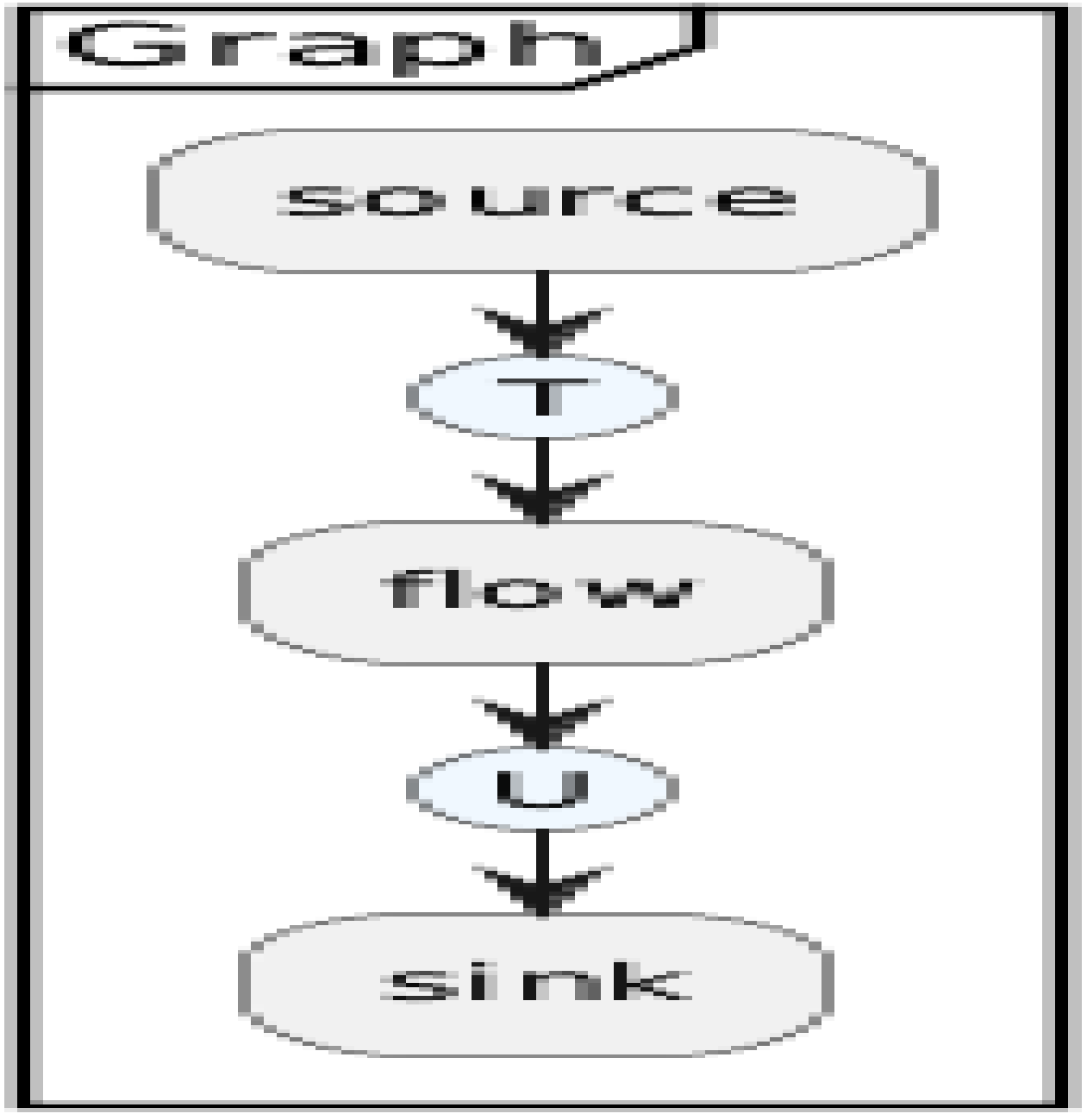
# Reactive streams

- Reactive streaming concept developed several times separately
  - *rxJava* (2014)
  - *Akka Streams* (2015)
  - *Project Reactor* (2015)
  - Many others
- Interoperability through `java.util.concurrent.Flow`
  - Low-level

# Pekko Streams

- Mature, large ecosystem of integrations
- Tight integration with other asynchronous concepts (e.g. actors)
- Tight integration with network I/O (streams of bytes)
- Well-documented API
- Many concepts presented here also apply to other libraries

# Pekko Streams concepts



# Pekko Streams concepts (contd.)

- *Graph* is a blueprint for a closed, finite network of stages, which communicate by streaming elements
- `GraphStage<S extends Shape>.` is one processing stage within a graph
  - taking elements in through zero or more Inlets
  - emitting through Outlets
- It's completely up to the stage when and how to respond to arriving elements
- All built-in graph stages embrace *backpressure* and *bounded processing*

# Pekko graph stage types

- `Source<T, M>`. has one outlet of type `T`
- `Sink<T, M>`. has one inlet of type `T`
- `Flow<A, B, M>`. has one inlet of type `A` and one outlet of type `B`
- `RunnableGraph<M>`. has no inlets or outlets

# Hello, streams

Let's do a hello world with some numbers:

```
ActorSystem system = ActorSystem.create("QuickStart");
Materializer materializer = ActorMaterializer.create(system);

Source<Integer, NotUsed> numbers = Source.range(1, 3);

Sink<Integer, CompletionStage<Done>> print =
    Sink.foreach(i -> System.out.println(i));

CompletionStage<Done> done = numbers.runWith(print, materializer);

done.toCompletableFuture().get();
```

# Stream materialization

- Graph is only a blueprint: nothing runs until it's given to a materializer, typically `ActorMaterializer`
- All graph stages are generic in their materialized type `M`
- Graph can be materialized more than once by calling `run()` or `runWith()`

```
class Source<T, M> {
  // A graph which materializes into the M2 of the sink (ignoring
  ↪  source's M)
  public RunnableGraph<M2> to(Sink<T,M2> sink);

  // Materializes, and returns the M of the sink (ignoring this source's
  ↪  M)
  public <M2> M2 runWith(Sink<T, M2> sink, Materializer m) { ... }

  // A graph which materializes into the result of applying [combine]
  ↪  to
  // this source's M and the sink's M2
  public <M2, MR> RunnableGraph<MR> toMat(Sink<T,M2> sink,
                                          Function2<M,M2,MR> combine);
}

class RunnableGraph<M> {
  public M run(Materializer m);
}
```

# Reusable pieces

- `Source`, `Sink` and `Flow` are all normal, immutable objects, so they're ideal to be constructed in reusable factory methods:

```
public Sink<String, CompletionStage<IOResult>> lineSink(String
↪  filename) {
  Sink<ByteString, CompletionStage<IOResult>> file =
  ↪  FileIO.toPath(Paths.get(filename));

  return Flow.of(String.class)
    .map(s -> ByteString.fromString(s + "\n"))
    .toMat(file, Keep.right());
}
```

Let's write our numbers to a file!

```
numbers
  .map(i -> i.toString())
  .runWith(lineSink("/tmp/numbers.txt"),
  ↪  materializer).toCompletableFuture().get();
```

# Time-based processing

```
Source<Integer, NotUsed> numbers = Source.range(1, 100000000);

Sink<Integer, CompletionStage<Done>> print =
    Sink.foreach(i -> System.out.println(i));

CompletionStage<Done> done = numbers
    .throttle(1, Duration.create(1, TimeUnit.SECONDS), 1,
              ThrottleMode.shaping())
    .runWith(print, materializer);
```

- This does what you expect: print one message per second
- No `OutOfMemoryError`, akka buffers only as needed: *backpressure*

# A directory crawler

- Let's put our `demo-project` to some good use
- Using asynchronous Java techniques (e.g. akka streams), write a program that concurrently:
  - Recursively looks at the contents of all files under the current path
  - Looks for lines matching `somekey=somevalue`
  - Prints them
- Then, extend your program to additionally:
  - Add them to our `demo-project`'s database by its REST API
    - `POST http://localhost:8080/values`
- Some tips:
  - List files: https://pekko.apache.org/docs/pekko-connectors/current/file.html#listing-directory-contents
  - Read a file's contents: https://pekko.apache.org/docs/pekko/current//stream/stream-io.html#streaming-file-io
  - Make http request
    - Plain Java: https://docs.oracle.com/en/java/javase/11/docs/api/java.net.http/java/net/http/HttpClient.html
    - Spring: https://docs.spring.io/spring-framework/docs/3.0.x/javadoc-api/org/springframework/web/client/RestTemplate.html
    - Pekko:https://pekko.apache.org/docs/pekko-http/current/client-side/request-level.html
  - Run a specific Java main class:

```
mvn exec:java -Dexec.mainClass="com.example.Main"
```

# Temperature check

| Name | @@@@@ I'm learning new stuff | @@@@@ I know microservices |
| --- | --- | --- |
| | | |

# Summary

- Systems architecture is about making choices
- Know at least two of each:
  - Programming language
  - System container setup
  - Program flow style
  - Data store
  - Message queue
  - Deployment tool
- Have periodic, and real, retrospectives in your team

# Thank you

Thank you for your participation!
Any questions or comments, at any time?
Jan Ypma jan@ypmania.net

https://world.hey.com/joaoqalves/disasters-i-ve-seen-in-a-microservices-world-a9137a51

https://copyconstruct.medium.com/testing-in-production-the-safe-way-18ca102d0ef1

- Orange post-its
- Blue, Purple, Pink, Green (same size)
- Yellow small (actor)
- Yellow big (aggregate)
- Markers
- Tape